SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I D Raghul 192511115 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

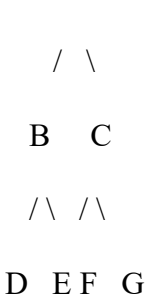
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



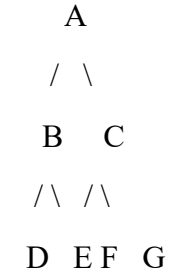
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

SIMATS ENGINEERING

ARTIFICIAL INTELLIGENCE — CSA17

CO2 DRY RUN EVALUATION

Assessment Tool 3 | Solved Answers

Student Name: _____ Register Number: _____

Department / Section: _____ Date: _____

Course Outcome CO2: Search techniques including BFS, DFS, A*, CSP, Minimax and Alpha-Beta pruning.

QUESTION 1 — BREADTH-FIRST SEARCH (BFS)

Given graph: A is the root; B and C are its children; D and E are children of B; F and G are children of C. Visit children from left to right.

BFS visits nodes level by level using a FIFO queue.

Iteration	Queue before expansion	Visited node	Queue after expansion
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

1. BFS traversal order: A → B → C → D → E → F → G

2. Queue after each iteration: [B,C], [C,D,E], [D,E,F,G], [E,F,G], [F,G], [G], [].

QUESTION 2 — DEPTH-FIRST SEARCH (DFS)

Using the same graph, DFS explores one branch as deeply as possible before backtracking. Assuming left-to-right traversal, the order is A, B, D, E, C, F, G.

Iteration	Stack (top at right)	Visited node	Stack after expansion
1	[A]	A	[C, B]
2	[C, B]	B	[C, E, D]
3	[C, E, D]	D	[C, E]
4	[C, E]	E	[C]
5	[C]	C	[G, F]
6	[G, F]	F	[G]
7	[G]	G	[]

DFS traversal order: A → B → D → E → C → F → G

Stack convention: The top is at the right. Push each node's children in reverse order (right child first) so that the left child is visited first.

QUESTION 3 — 8-PUZZLE USING BFS

Initial state

1	**3**	**6**
5	**0**	**2**
4	**7**	**8**

Goal state

1	**2**	**3**
4	**5**	**6**
7	**8**	**0**

BFS expands states in increasing depth order. The blank (0) can move up, down, left or right when legal. The table below uses the move order Up, Down, Left, Right and avoids adding duplicate states already discovered.

Iter.	Current state expanded	Queue before expansion	Queue after expansion
1	1 3 6/5 0 2/4 7 8	[]	[1 0 6/5 3 2/4 7 8; 1 3 6/5 7 2/4 0 8; 1 3 6/0 5 2/4 7 8; 1 3 6/5 2 0/4 7 8]
2	1 0 6/5 3 2/4 7 8	[1 3 6/5 7 2/4 0 8; 1 3 6/0 5 2/4 7 8; 1 3 6/5 2 0/4 7 8]	[1 3 6/5 7 2/4 0 8; 1 3 6/0 5 2/4 7 8; 1 3 6/5 2 0/4 7 8; 0 1 6/5 3 2/4 7 8; …]
3	1 3 6/5 7 2/4 0 8	[1 3 6/0 5 2/4 7 8; 1 3 6/5 2 0/4 7 8; 0 1 6/5 3 2/4 7 8; 1 6 0/5 3 2/4 7 8]	[1 3 6/0 5 2/4 7 8; 1 3 6/5 2 0/4 7 8; 0 1 6/5 3 2/4 7 8; 1 6 0/5 3 2/4 7 8; …]
4	1 3 6/0 5 2/4 7 8	[1 3 6/5 2 0/4 7 8; 0 1 6/5 3 2/4 7 8; 1 6 0/5 3 2/4 7 8; 1 3 6/5 7 2/0 4 8; …]	[1 3 6/5 2 0/4 7 8; 0 1 6/5 3 2/4 7 8; 1 6 0/5 3 2/4 7 8; 1 3 6/5 7 2/0 4 8; …]
5	1 3 6/5 2 0/4 7 8	[0 1 6/5 3 2/4 7 8; 1 6 0/5 3 2/4 7 8; 1 3 6/5 7 2/0 4 8; 1 3 6/5 7 2/4 8 0; …]	[0 1 6/5 3 2/4 7 8; 1 6 0/5 3 2/4 7 8; 1 3 6/5 7 2/0 4 8; 1 3 6/5 7 2/4 8 0; …]

Important note on the provided task: the worksheet asks for only five iterations, but also asks for the complete solution path to the goal. The goal is not reached in the first five expansions. Therefore, the five-row table is the requested partial dry run; the full shortest path is given below.

Questions and answers

1. Which node is expanded in each iteration? Iteration 1: 1 3 6/5 0 2/4 7 8; Iteration 2: 1 0 6/5 3 2/4 7 8; Iteration 3: 1 3 6/5 7 2/4 0 8; Iteration 4: 1 3 6/0 5 2/4 7 8; Iteration 5: 1 3 6/5 2 0/4 7 8.

2. How many states are generated in total? BFS generates states as it explores; the total depends on whether 'generated' means successor states considered or unique states enqueued. In the five-row dry run, 12 new unique states are enqueued during those expansions (not counting the initial state).

3. Complete solution path: The shortest path contains 8 moves and 9 states.

Step	State	Move from previous state
0	1 3 6/5 0 2/4 7 8	Initial state
1	1 3 6/5 2 0/4 7 8	Right
2	1 3 0/5 2 6/4 7 8	Up
3	1 0 3/5 2 6/4 7 8	Left
4	1 2 3/5 0 6/4 7 8	Down
5	1 2 3/0 5 6/4 7 8	Left
6	1 2 3/4 5 6/0 7 8	Down
7	1 2 3/4 5 6/7 0 8	Right
8	1 2 3/4 5 6/7 8 0	Right

4. Why does BFS find the shortest solution in an unweighted 8-puzzle? BFS explores all states at depth 0, then depth 1, depth 2, and so on. When each move has equal cost, the first time BFS reaches the goal it has found

a path with the fewest moves.